

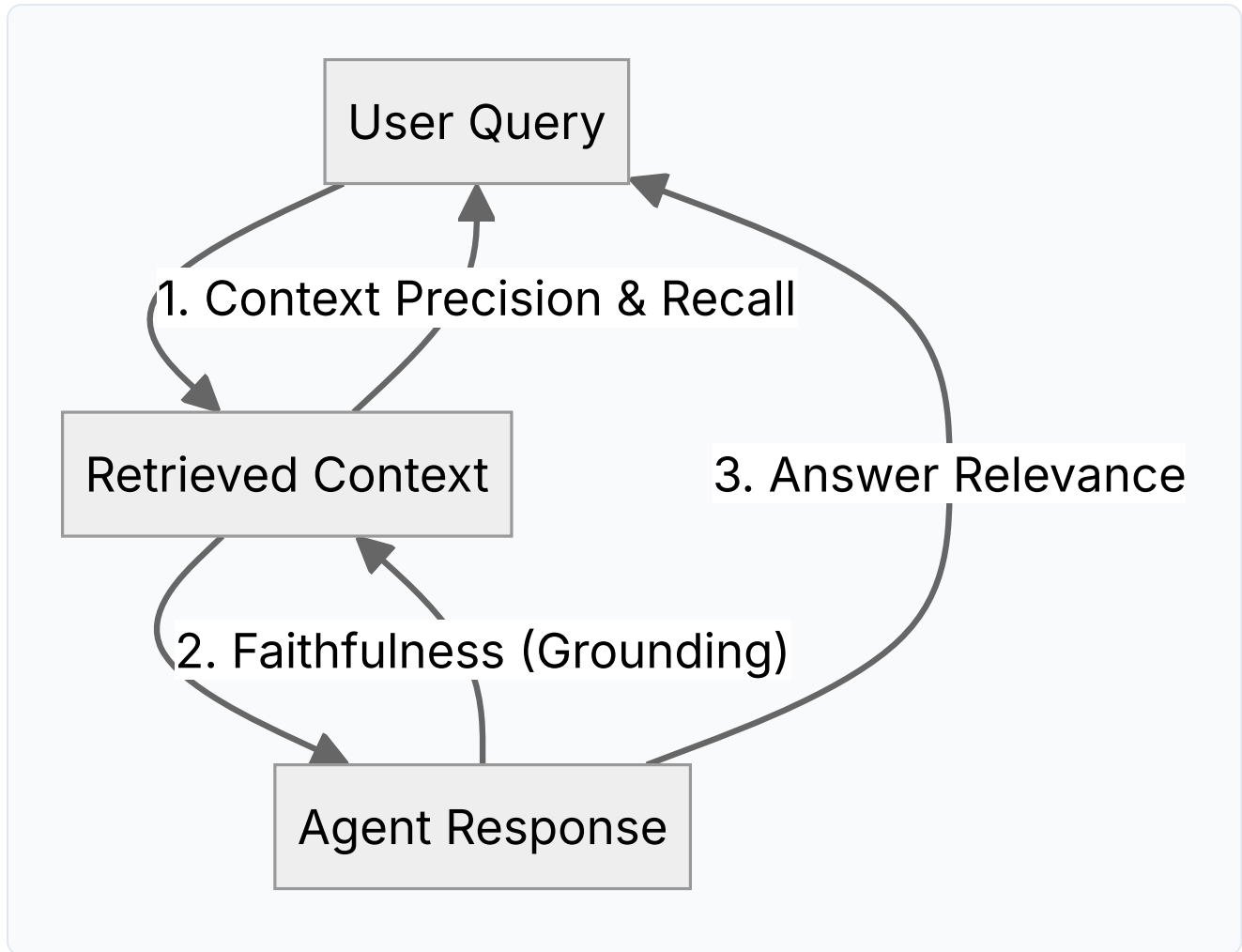
Chapter 11: Evaluation Harnesses, LLM-As-A-Judge, & CI/CD

The Fallacy of the "Vibe Check" Most engineering teams evaluate AI agents through "vibe checks": an engineer tweaks a system prompt, runs 3 manual queries in terminal, says *"looks pretty good"*, and merges to production. Two days later, customer support is overwhelmed because the modified prompt degraded accuracy on edge cases by 28%.

You cannot improve what you cannot measure. In deterministic software, unit tests check `assert add(2, 2) == 4`. In probabilistic software, you need **Automated Evaluation Harnesses** and **Semantic Assertions** integrated into CI/CD.

1. The Four Pillars of RAG Evaluation (RAG Triad)

When evaluating retrieval-augmented generation and reasoning agents, we isolate three components: **The Query**, **The Retrieved Context**, and **The Generated Response**.



The 4 Core Metrics Defined

1. **Faithfulness (Groundedness):** Measures whether every factual claim in the response is strictly supported by the retrieved context. (Prevents hallucinations).

$$\text{Faithfulness} = \frac{|\text{Supported Claims in Response}|}{|\text{Total Claims in Response}|}$$

2. **Answer Relevance:** Measures how directly the response addresses the user's original query, penalizing redundant or evasive answers.
3. **Context Recall:** Measures whether the retriever fetched all facts necessary to produce the ground-truth answer.
4. **Context Precision:** Measures the signal-to-noise ratio in retrieval. Are the most relevant chunks ranked at the top ($Rank = 1, 2$) rather than buried at the bottom?

2. Building an Automated "LLM-As-A-Judge" Evaluator from Scratch

We don't need heavy proprietary SaaS platforms to run evaluations. We can build an exact, reproducible evaluation judge in Python using structured outputs:

3. Creating a Golden Benchmark Evaluation Dataset

To prevent regressions, maintain a `golden_dataset.json` containing 50–200 challenging production queries with known ground truths:

4. Integrating the Eval Suite into CI/CD (GitHub Actions)

Never allow an engineer to merge a prompt update or change an embedding model without running the automated evaluation test runner:

Now, any prompt modification that introduces subtle hallucinations or degrades retrieval quality is blocked at the Pull Request stage before reaching a single customer!

4. Runnable Model: Can You Trust the Judge? Agreement, Bias and a CI Gate

An LLM judge is a measurement instrument, so calibrate it before trusting a score.

Agreement with humans beyond chance (Cohen's kappa)

Raw agreement flatters a judge when one label dominates; kappa subtracts the agreement you would get by guessing. A common reading: below 0.4 is weak, 0.6 to 0.8 is substantial.

Position bias: does the verdict change when you swap the answers?



The second judge passes the swap test and is still useless: **position consistency is necessary, not sufficient**. Also test length bias (does a padded answer win?) and self-preference (does a model prefer its own outputs?).

A CI gate on a lower confidence bound, not the raw pass rate



The gate encodes the right question: not "did the score exceed the threshold?" but "are we confident the true pass rate does?" Small test sets need a higher raw score to pass, which pushes teams to grow the set.

Evaluation pipeline checklist

1. A **golden set** with human-verified labels, versioned with the code.
2. **Deterministic checks first** (schema, exact match, tool-call correctness), judge-model scoring only for what code cannot decide.
3. **Calibrate the judge** (kappa against human labels, swap test, length test) whenever the judge model or rubric changes.
4. **Run on every change** that can affect behaviour: prompt, model, tool, retrieval index.
5. **Report with uncertainty** (confidence interval), and track cost and latency next to quality.

Further Reading

- [Judging LLM-as-a-judge \(MT-Bench\)](#)
- [Eugene Yan: LLM evaluators](#)
- [lm-evaluation-harness](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. Why calibrate an LLM judge against human labels?

To measure its agreement and error directions (TPR/FPR) before trusting its scores.

▼ 2. What biases do LLM judges show?

Position, verbosity, self-preference and formatting bias.

▼ **3.** How do you put evals in CI?

Run a fixed task suite on each change, compare with the baseline within confidence intervals, and block regressions.

▼ **4.** What is pass^k ?

The probability that all k independent trials succeed; it measures consistency.